

Relazione - Versione 2: Modello neurale con NumPy

Introduzione

La Versione 2 estende il modello linguistico a caratteri sostituendo il conteggio diretto delle transizioni con un piccolo modello neurale implementato in NumPy.

Il corpus, il vocabolario, il compito basato su coppie di caratteri e il ciclo di generazione autoregressiva restano invariati. In questo modo viene isolata la nuova idea: le probabilità sono ora prodotte da parametri addestrabili.

```
Caratteri
↓
Identificatori interi
↓
Input one-hot
↓
Pesi addestrabili
↓
Probabilità softmax
↓
Testo generato
```

Obiettivo della Versione 2

Il modello deve essere in grado di:

- riutilizzare il corpus e le coppie di caratteri della versione precedente;
- convertire i caratteri in identificatori interi e vettori one-hot;
- rappresentare le previsioni con una matrice di pesi addestrabile;
- convertire i logits in probabilità tramite softmax;
- misurare l'errore di previsione con la cross-entropy;
- apprendere i parametri tramite gradient descent;
- generare testo riproducibile dalle probabilità apprese.

La versione resta volutamente piccola, così che ogni calcolo possa essere ispezionato direttamente.

Architettura

Il flusso completo della Versione 2 è:

```
Corpus
↓
Vocabolario e ID dei caratteri
↓
Coppie input-target adiacenti
↓
Matrice di input one-hot
↓
Matrice di pesi addestrabile
↓
Logits e softmax
↓
Cross-entropy e gradient descent
↓
Generatore autoregressivo
```

Il contesto contiene ancora un solo carattere. Il carattere appena campionato diventa il contesto per il passo di generazione successivo.

Rappresentazione numerica

I calcoli neurali operano su numeri. A ciascuno dei 27 caratteri del vocabolario viene assegnato un identificatore intero.

```
model
↓
[15, 17, 7, 8, 14]

m -> o diventa 15 -> 17
o -> d diventa 17 -> 7
d -> e diventa 7 -> 8
e -> l diventa 8 -> 14
```

Il corpus contiene 440 caratteri e produce 439 esempi input-target adiacenti.

Input one-hot

Ogni ID di input seleziona un vettore one-hot di lunghezza 27: una posizione vale 1 e tutte le altre valgono 0.

Forma della matrice di input: (439, 27)

Matrice di pesi addestrabile

Il modello contiene una matrice di pesi 27 per 27. Ogni riga corrisponde a un carattere corrente e ogni colonna a un possibile carattere successivo.

Forma della matrice dei pesi: (27, 27)

Parametri addestrabili: 729

Moltiplicare un input one-hot per la matrice seleziona la riga associata al carattere corrente. I 27 valori risultanti sono i logits.

I pesi iniziali sono piccoli valori casuali generati con un seed NumPy locale pari a 42. Questo rende riproducibile l'intera esecuzione.

Probabilità softmax

I logits sono punteggi senza vincoli. Softmax li converte in una distribuzione di probabilità sull'intero vocabolario.

```
shifted = logits - max(logits)
probabilities = exp(shifted) / sum(exp(shifted))
```

Sottrarre il logit massimo migliora la stabilità numerica senza modificare le probabilità risultanti.

- ogni probabilità è compresa tra 0 e 1;
 - le 27 probabilità di uno stesso contesto sommano a 1;
 - logits più alti producono probabilità maggiori.
-

Che cosa cambia

Il modello statistico calcolava le probabilità direttamente dai conteggi osservati. Il modello neurale le produce tramite parametri che devono essere appresi.

Cross-entropy loss

La cross-entropy misura quanta probabilità il modello assegna al carattere successivo corretto.

```
loss = -mean(log(probabilità del target corretto))
```

Una probabilità alta per il target corretto produce una loss bassa. Una probabilità bassa produce una loss alta.

Gradient descent

Ogni passo di training esegue quattro operazioni:

- calcola logits e probabilità per tutti i 439 esempi;
- misura la cross-entropy loss media;
- calcola i gradienti dei logits e dei pesi;
- aggiorna i pesi nella direzione che riduce la loss.

```
gradiente logits = probabilità - target
gradiente pesi = inputs.T @ gradiente logits
weights -= learning rate * gradiente pesi
```

Il training usa full-batch gradient descent con learning rate 20.0 e 1.000 aggiornamenti. I calcoli sono implementati direttamente in NumPy, senza differenziazione automatica.

Risultato del training

```
Loss iniziale: 3.297271
Loss finale: 1.946110
```

La riduzione regolare della loss mostra che la matrice ha appreso le transizioni tra caratteri presenti nel corpus.

Probabilità apprese

Dopo il training, una riga della matrice dei pesi può essere ispezionata come distribuzione del carattere successivo per un determinato contesto.

Per il carattere `m`, le probabilità più alte sono:

```
a: 0.2212 e: 0.2212 o: 0.2212
spazio: 0.1101 p: 0.1101 s: 0.1101
```

Questi valori riproducono da vicino la distribuzione empirica delle transizioni. Softmax assegna inoltre una probabilità molto piccola ma non nulla alle transizioni mai osservate.

Generazione neurale del testo

La generazione conserva lo stesso ciclo autoregressivo usato in precedenza:

- converte il carattere corrente nel suo ID intero;
- seleziona i logits addestrati e applica softmax;
- campiona il prossimo ID dalla distribuzione di probabilità;
- riconverte l'ID campionato in un carattere;
- usa quel carattere come contesto successivo.

Un seed NumPy locale rende riproducibile la dimostrazione di 300 caratteri.

Controlli inclusi

Le asserzioni del notebook verificano:

- numero e forma corretti degli esempi numerici;
- vettori one-hot validi e dimensioni dei parametri;
- una loss finale inferiore a quella iniziale;
- la somma delle probabilità del carattere successivo uguale a 1;
- la riproducibilità e la lunghezza richiesta della generazione.

L'esecuzione completa termina con il messaggio `All checks passed.`

Cosa insegna la Versione 2

La progressione centrale è:

codificare -> prevedere -> misurare -> derivare -> aggiornare ->
campionare

La Versione 2 mostra come una distribuzione di probabilità possa essere appresa tramite ottimizzazione invece di essere calcolata direttamente dai conteggi.

Interpretazione

Il modello è neurale perché le previsioni derivano da parametri addestrabili ottimizzati tramite gradient descent. Resta un modello bigram a caratteri perché usa come contesto un solo carattere precedente.

La capacità espressiva resta simile a quella del modello statistico. Il progresso è l'apprendimento dei parametri, non ancora una migliore qualità linguistica a lungo termine.

Limitazioni

- il contesto contiene un solo carattere;
 - il corpus è piccolo e incorporato nel notebook;
 - training e ispezione utilizzano gli stessi esempi;
 - tutti gli esempi sono elaborati insieme in un unico full batch;
 - non sono presenti hidden layer, embedding, differenziazione automatica o framework;
 - il testo generato mostra regolarità locali ma non coerenza a lungo termine.
-

Conclusione

La Versione 2 è il primo modello neurale addestrabile di Building LLM. Converte i caratteri in input numerici, apprende una matrice di pesi con NumPy e genera testo dalle distribuzioni di probabilità risultanti.